

# 长序列 Attention 机制更新与 CGC

---

## 一、问题背景

## 二、GSU 和 ESU 是什么

1. GSU: 从超长历史里粗筛 Top-K
2. ESU: 对 Top-K 历史做精细 attention

## 三、Q、K、V 在推荐里分别是什么

## 四、FFN 是什么

1.  $Q = \text{FFN}(\text{target\_embedding})$
2.  $Q_{\text{next}} = \text{FFN}(\text{attention\_output})$

## 五、SWiGLU 是什么

## 六、STCA 多层堆叠怎么做

做几层合适

## 七、KV 复用和 KV 缓存

1. LLM 中的 KV cache
2. MQA/GQA 中的 KV 共享
3. 推荐系统里的 KV 复用

## 八、为什么共现序列可能变差

## 九、注意力分布熵怎么做

## 十、CGC 和 PLE 是什么关系

## 十一、作者这个模型可能是什么结构

## 十二、实践建议

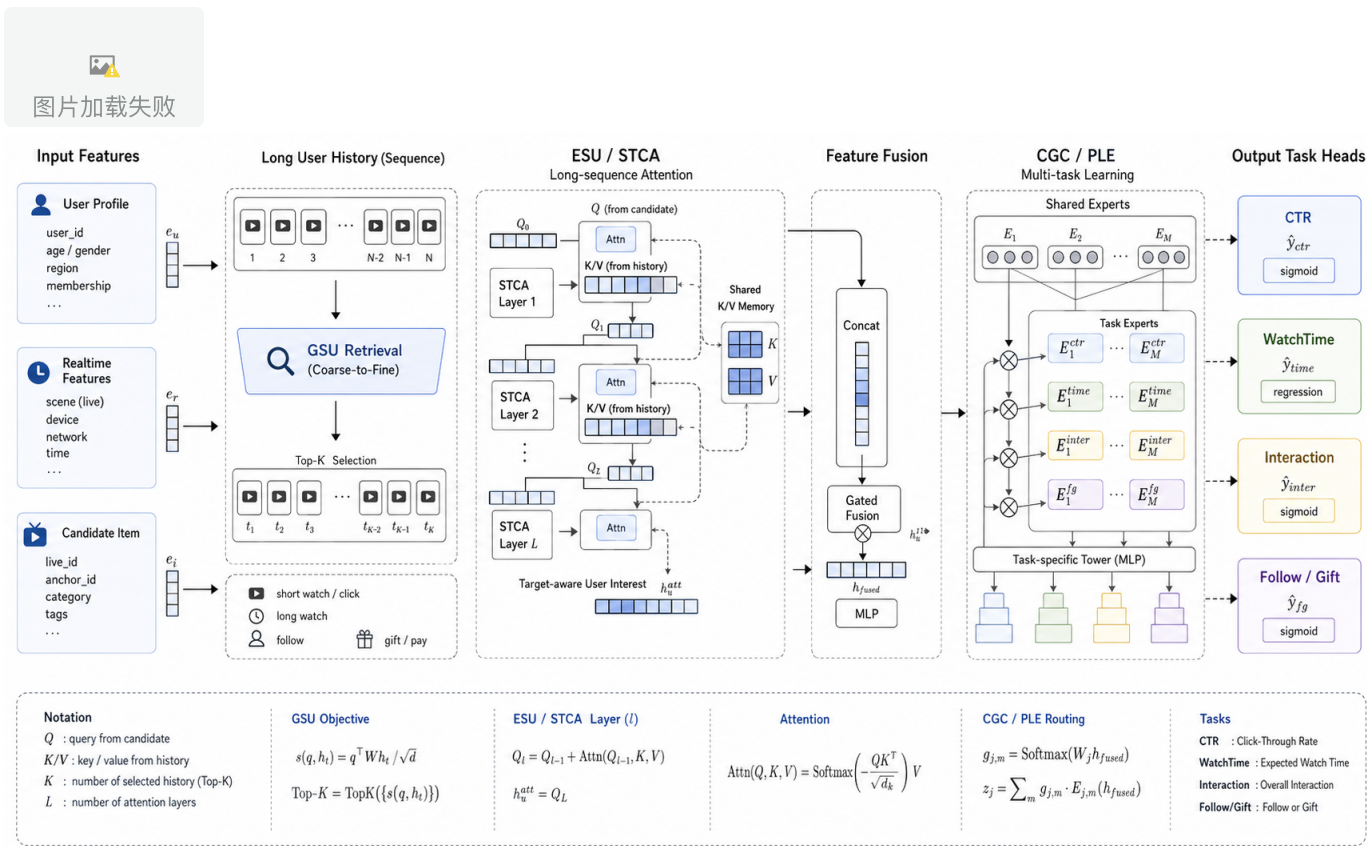
1. 序列扩长前先证明有价值
2. STCA 优先用于真实用户行为序列
3. 层数从 2 层开始
4. KV 复用要工程化实现
5. CGC/PLE 要看任务冲突

## 十三、实验与监控清单

## 十四、总结

## 参考资料

# 一、问题背景



**读图说明：** 这张图是完整精排模型视角。长历史序列先进入 **GSU** 做粗筛，得到 Top-K 相关历史；然后进入 **ESU/STCA** 做多层 target attention，输出 target-aware user interest。该兴趣表达和其他 dense/sparse 特征融合后，进入 **CGC/PLE** 多任务模块，分别服务 CTR、WatchTime、Interaction、Follow/Gift 等任务。

原模型中的用户序列长度为 **x**。业务观察后发现两个现象：

- 1

1. 大量用户存在兴趣偏移：更早的观播记录和最近的 item 类别有明显差异
- 2

2. 很多用户一周内观看时长大于 10s 的记录数超过 x 条

这说明原来的短序列不能完全表达用户兴趣。用户兴趣不是一个静态点，而是分层的：

- 1

短期兴趣：最近几次点击、最近一个 session、当前上下文
- 2

中期兴趣：最近几天反复出现的类目和主播
- 3

长期兴趣：稳定偏好的内容类型、消费习惯和付费倾向

因此作者把序列长度从 **x** 扩大到 **y**。但序列变长后，普通 target attention 会成为瓶颈：计算更贵、噪声更多、一层 attention 的表达能力不足。于是引入了基于 GSU + ESU 的长序列建模，并在 ESU 内部升级为 STCA 结构。

整体思路可以概括为：

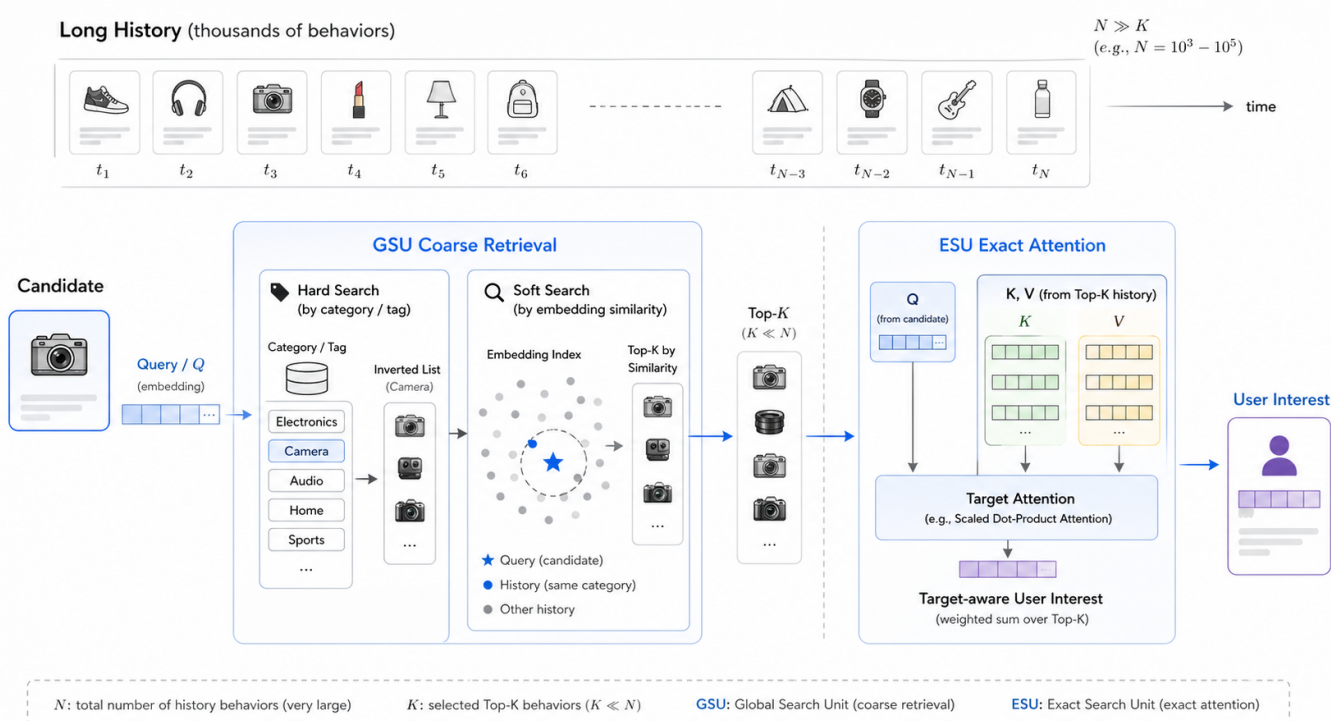
- 1 长序列扩容
- 2 => GSU 粗筛相关历史
- 3 => ESU/STCA 精细 target attention
- 4 => CGC/PLE 多任务融合
- 5 => CTR、时长、互动、关注等任务输出

## 二、GSU 和 ESU 是什么

1. 精排中做GSU为了降低线上推理耗时，需要先将部分历史序列建成倒排索引，一般用redis这一类kv数据库做
2. GSU中可以用hard search（规则）和soft search等方式，ESU的Q就是当前样本item的表示



图片加载失败



**读图说明：** 图左侧是超长用户历史序列和当前候选 item。中间的 **GSU Coarse Retrieval** 包括 Hard Search 和 Soft Search：Hard Search 可以按类目、主播、标签等规则召回，Soft Search 可以用 embedding 相似度召回。GSU 输出 Top-K 历史后，右侧 **ESU Exact Attention** 使用 Q/K/V target attention 得到用户对当前候选的兴趣表达。

GSU/ESU 最典型的来源是阿里 SIM：Search-based Interest Model。

它的核心问题是：用户历史行为很长，不能直接把几千甚至几万条历史全部放进复杂 attention。于是分成两步：

- 1 GSU: General Search Unit, 粗筛
- 2 ESU: Exact Search Unit, 精排 attention

## 1. GSU: 从超长历史里粗筛 Top-K

GSU 的目标是：从很长的用户历史行为中，快速找出和当前候选 item 最相关的一小段历史。

例如：

- 1 用户历史行为：10000 条
- 2 当前候选直播间：游戏类直播间
- 3 GSU 输出：Top-K 条游戏、相似主播、相似内容相关的历史观播记录

常见 GSU 方式包括：

- 1 Hard Search: 按类目、主播、标签、行为类型等规则匹配
- 2 Soft Search: 用 embedding 内积、ANN 检索、向量相似度召回历史行为
- 3 Hybrid Search: 规则召回 + 向量召回混合

GSU 解决的是：

- 1 序列太长，算不过来

## 2. ESU: 对 Top-K 历史做精细 attention

ESU 接收 GSU 筛出来的 Top-K 历史行为，然后用 target attention 做精细建模：

- 1  $Q = \text{candidate embedding}$
- 2  $K, V = \text{Top-K history behavior embeddings}$
- 3  $\text{attention}(Q, K, V)$
- 4  $\Rightarrow \text{target-aware user interest}$

ESU 解决的是：

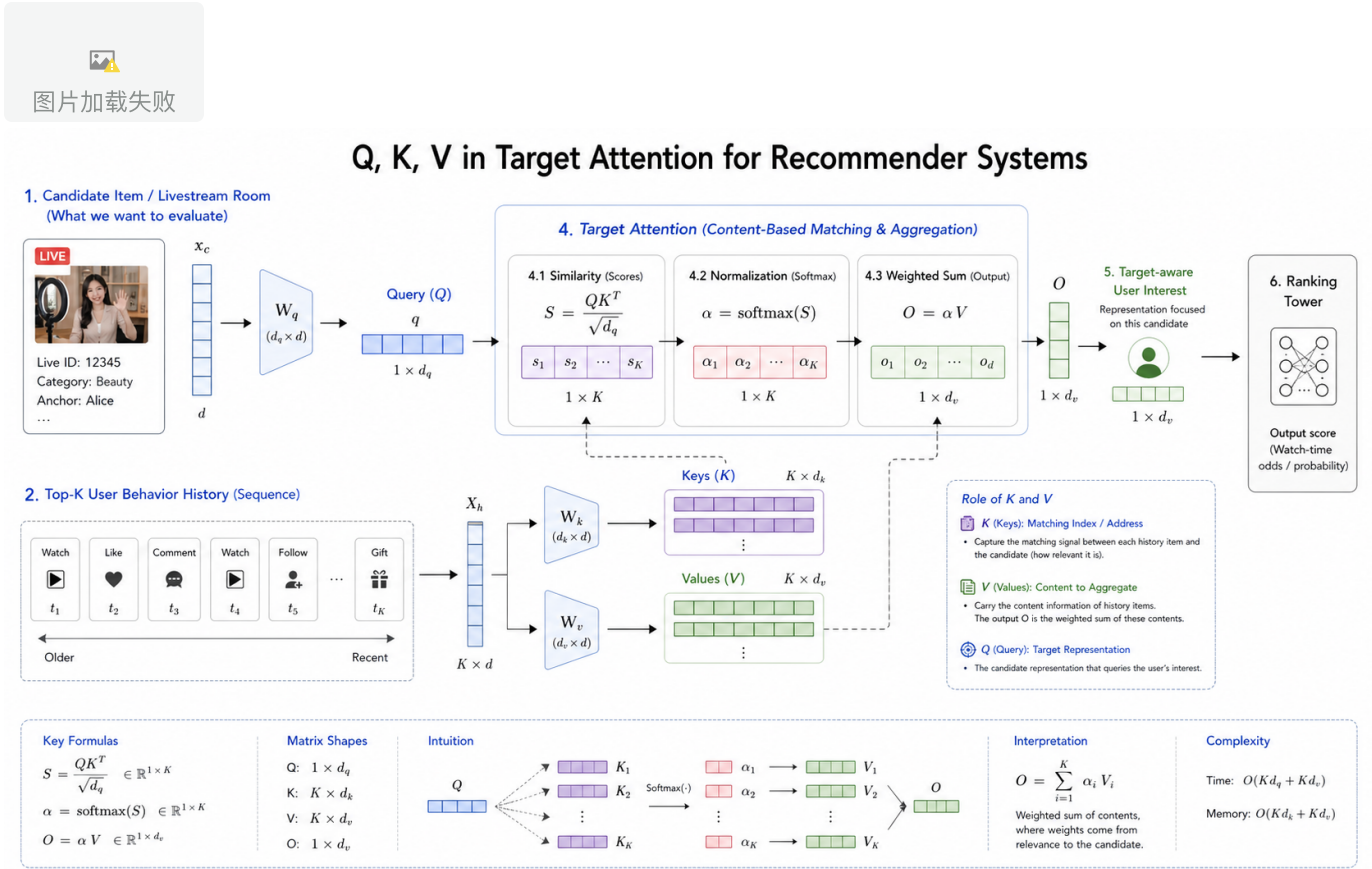
- 1 粗筛出来的历史里，哪些行为真正和当前候选相关

可以把 GSU/ESU 理解成：

- 1 GSU: 先把图书馆里可能相关的书找出来
- 2 ESU: 再逐页精读其中最相关的内容



三、Q、K、V 在推荐里分别是什么



**读图说明：** 这张图把 Q/K/V 的角色拆开讲。候选直播间经过  $W_q$  生成 Query；Top-K 用户历史经过  $W_k$  和  $W_v$  分别生成 Keys 和 Values。Attention block 先计算  $QK^T/\sqrt{d}$  得到相关性分数，再 softmax 成权重，最后对 V 加权求和得到 target-aware user interest。K 更像匹配索引，V 更像被聚合的内容。

在 target attention 场景中，通常是：

- 1

Q：当前候选 item / 候选直播间，以及item相关的特征
- 2

K：用户历史行为序列，用来匹配相关性
- 3

V：用户历史行为序列，用来聚合内容

K 和 V 通常来自同一批历史行为，但不是完全一样的东西。一般会通过不同参数投影：

- 1

$K = \text{history\_embedding} @ W_k$
- 2

$V = \text{history\_embedding} @ W_v$

直观理解：

- 1

K 是地址 / 索引：用来判断这个历史行为和候选有多相关
- 2

V 是内容 / 信息：相关之后真正被加权汇总的信息

例如历史行为是：

- 1 用户观看某游戏主播 120s, 并发生点赞

K 侧可能更强调：

- 1 类目、主播、内容语义, 用于和 candidate 匹配

V 侧可能保留：

- 1 观看时长、互动强度、行为类型、时间衰减后的兴趣信息

候选 item 从哪里来？

推荐链路通常是：

- 1 召回阶段生成几百/几千个候选直播间
- 2 粗排筛到几十/几百个
- 3 精排对每个 candidate 分别打分

对每个 candidate, 都构造一个 Q, 然后去查询同一个用户历史 K/V。

## 四、FFN 是什么

FFN 是 Feed Forward Network, 前馈网络。

Transformer 中常见 FFN 是：

- 1  $FFN(x) = W2 \cdot \text{activation}(W1 \cdot x + b1) + b2$

常见结构是：

- 1 Linear(d -> 4d)
- 2 GELU / ReLU
- 3 Linear(4d -> d)

在 target attention 中, 有两个容易混淆的写法。

### 1. $Q = FFN(\text{target\_embedding})$

这是生成初始 query:

```
1 target_embedding
2 => FFN
3 => Q0
```

意思是：只根据当前候选 item 生成查询向量。

## 2. $Q_{next} = FFN(attention\_output)$

这是根据上一层 attention 从历史里提取到的信息，生成下一层 query：

```
1 Q0 attend history => 01
2 01 => FFN => Q1
```

区别是：

```
1 Q0: 只知道当前候选
2 Q1: 已经融合过一轮历史兴趣
```

更稳妥的工程写法通常是：

```
1 Q_next = FFN(concat(Q_old, attention_output, target_embedding))
```

这样不会丢掉原始 target 信息。

## 五、SWiGLU 是什么

SWiGLU 一般读作：

```
1 Swi-GLU
```

中文可以念作：

```
1 斯维格鲁
```

它来自 Shazeer 的 GLU Variants Improve Transformer。SWiGLU 是一种带门控的前馈结构。

普通 FFN 可以理解为：

```
1 FFN(x) = W2 · GELU(W1x)
```

SWiGLU 可以写成：

$$1 \quad \text{SwiGLU}(x) = (\text{SiLU}(xW_1) \odot xW_2) W_3$$

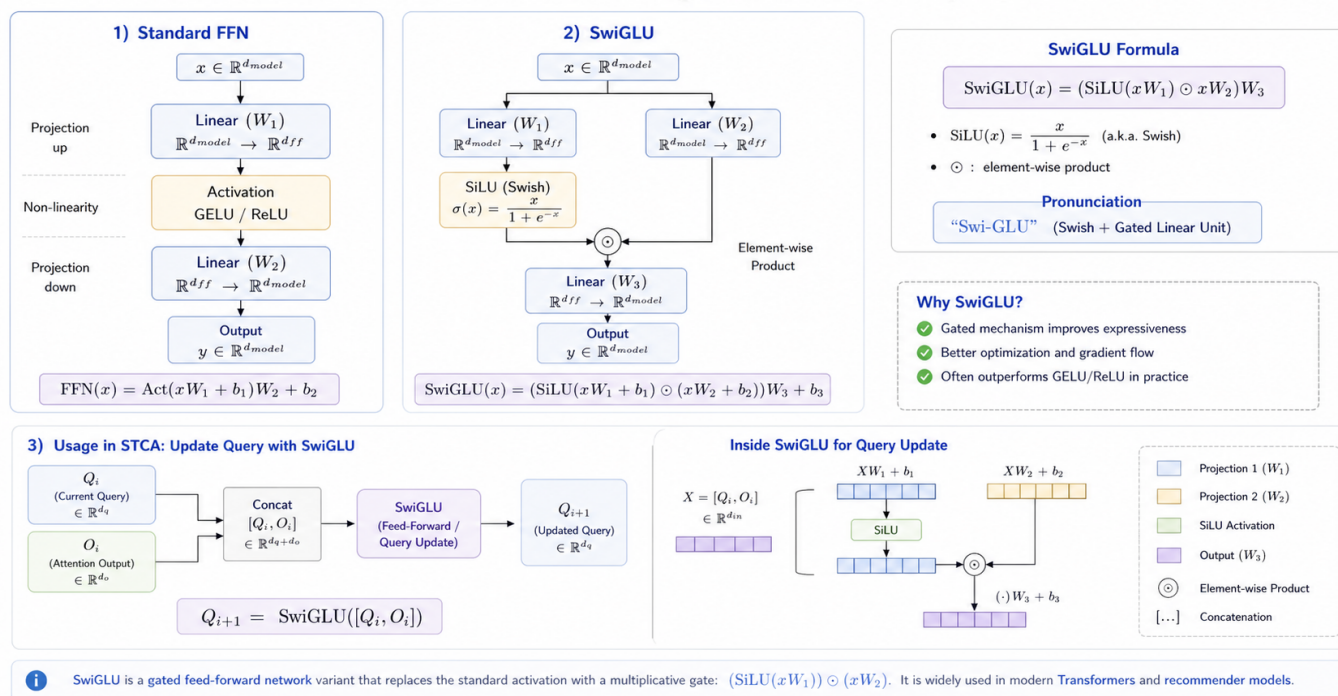
其中：

- 1  $\text{SiLU}(x) = x \cdot \text{sigmoid}(x)$
- 2  $\odot$  表示逐元素乘法



图片加载失败

## FFN vs SwiGLU and Usage in STCA (Query Update)



**i** SwiGLU is a **gated feed-forward network** variant that replaces the standard activation with a multiplicative gate:  $(\text{SiLU}(xW_1)) \odot (xW_2)$ . It is widely used in modern **Transformers** and **recommender models**.

**读图说明：** 这张图对比普通 FFN 和 SWiGLU。左侧普通 FFN 是 **Linear -> Activation -> Linear**；中间 SWiGLU 通过两路线性投影和逐元素乘法形成门控，公式为  $\text{SwiGLU}(x) = (\text{SiLU}(xW_1) \odot xW_2)W_3$ 。下方展示它在 STCA 中的用法：把当前 Query  $Q_i$  和 attention 输出  $O_i$  拼接后，经 SWiGLU 更新为下一层 Query  $Q_{i+1}$ 。这就是作者说“SWiGLU 替换 FFN 映射 Q”的具体含义。

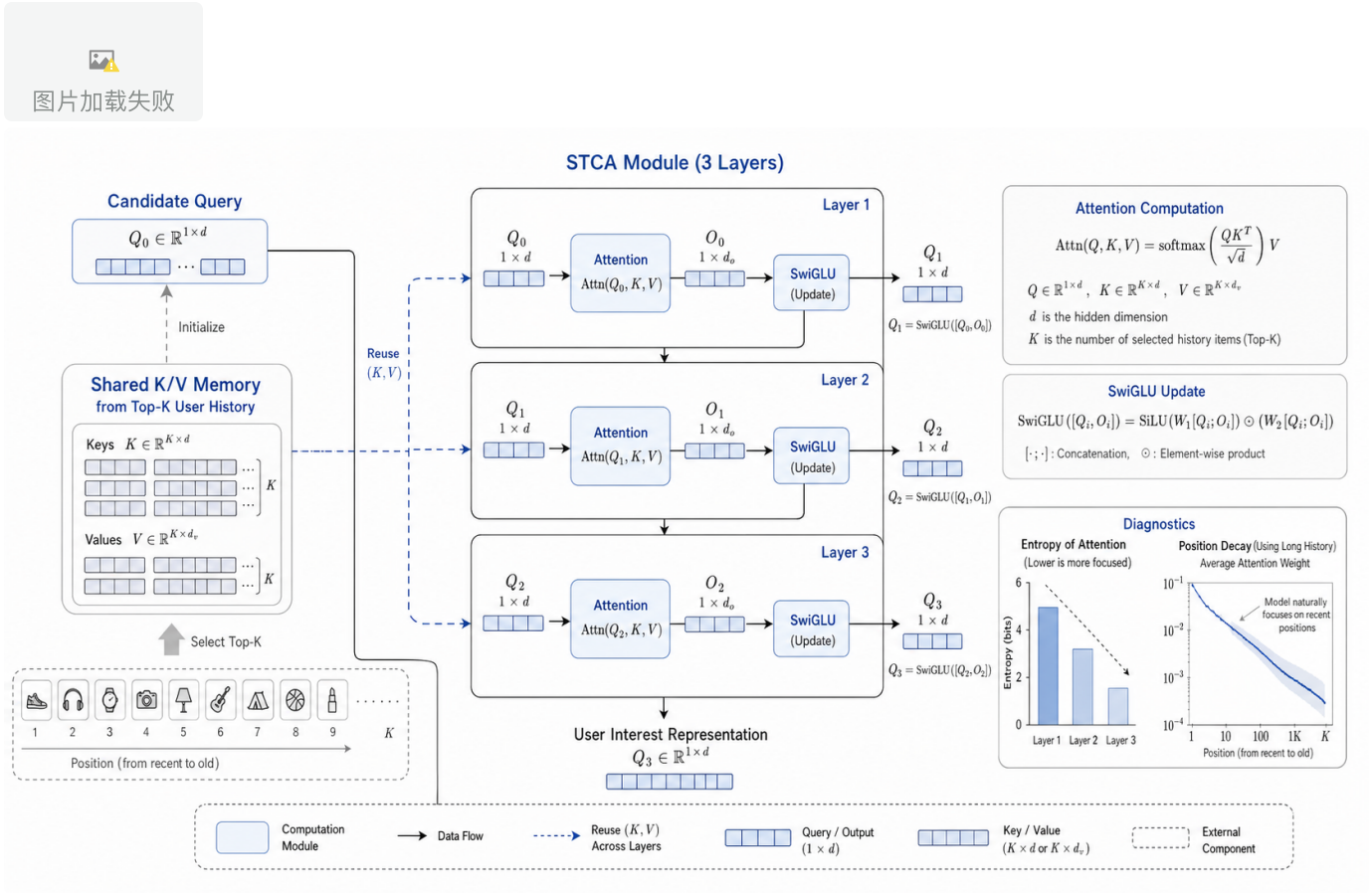
直观理解：

- 1 一支负责生成内容
- 2 一支负责生成门控
- 3 门控决定哪些维度被放大，哪些维度被压制

在作者这个场景里，用 SWiGLU 更新 Q，相当于让模型更灵活地决定：

- 1 下一层应该从什么视角继续查询用户历史序列

# 六、STCA 多层堆叠怎么做



**读图说明：** 这张图是 STCA 的核心。底部是 **Shared K/V Memory**，来自 GSU 筛选后的 Top-K 用户历史。上方是多层 **STCA Layer**，每层都使用同一份 K/V，但 Q 会逐层演化： **$Q_0 \rightarrow Q_1 \rightarrow Q_2$** 。每层先做  **$\text{Attention}(Q_i, K, V)$**  得到  **$O_i$** ，再通过  **$\text{SwiGLU}([Q_i, O_i])$**  生成下一层 Q。

根据作者描述，STCA 有三个关键点：

- 1. 用 SwiGLU 替换传统 FFN 映射 Q
- 2. 堆叠多层 target attention
- 3. K/V 保持复用，每层只更新 Q

可以理解为：对同一段用户历史，用多个逐层演化的 query 反复检索。

简化伪代码：

```

1  K = W_k(history_seq)
2  V = W_v(history_seq)
3
4  Q = swiglu(W_q(target_item))
5  outputs = []
6
7  for layer in range(num_layers):
8      score = torch.matmul(Q, K.transpose(-1, -2)) / math.sqrt(d)
9      attn = torch.softmax(score + mask, dim=-1)
10     O = torch.matmul(attn, V)
11
12     Q = q_update(torch.cat([Q, swiglu(O)], dim=-1))
13     outputs.append(O)
14
15 interest = torch.cat(outputs, dim=-1)

```

直觉上：

- 1 第一层：广泛查找和 target 相关的历史行为
- 2 第二层：基于第一层结果，进一步筛选更相关的兴趣
- 3 第三层：形成更聚焦、更高阶的 target-aware 兴趣表达

## 做几层合适

工业上建议从小到大试：

- 1 1 层：普通 target attention baseline
- 2 2 层：第一版 STCA 增强，性价比通常最高
- 3 3 层：长序列明显有效时尝试
- 4 4 层：需要强收益证明，注意 RT 和稳定性
- 5 超过 4 层：一般不建议作为第一阶段方案

判断是否值得加层，要看：

- 1 AUC / NLL / GAUC 提升
- 2 线上 RT 是否可接受
- 3 注意力熵是否逐层合理下降
- 4 长序列截断消融是否证明收益来自远端历史

## 七、KV 复用和 KV 缓存

最近很多论文都在讲 KV cache、KV reuse、KV sharing，但不同语境含义不同。



## 1. LLM 中的 KV cache

大语言模型自回归生成时，每生成一个 token，都需要 attend 前面的 token。

KV cache 做的是：

- 1 把历史 token 的 K/V 存起来
- 2 下一步只计算新 token 的 Q/K/V
- 3 旧 K/V 直接复用

它主要解决推理时重复计算问题。

## 2. MQA/GQA 中的 KV 共享

Multi-Query Attention / Grouped-Query Attention 做的是：

- 1 多个 Q head 共享更少的 K/V head

它主要减少显存和内存带宽。

## 3. 推荐系统里的 KV 复用

在作者这个场景里，KV 基本可以理解为用户历史序列的 key/value 表示：

- 1  $K = W_k(\text{history})$
- 2  $V = W_v(\text{history})$

复用体现在两层。

第一，同一个用户请求下，多个候选直播间共享同一份历史 K/V：

- 1 candidate A:  $Q_A$  attend K,V
- 2 candidate B:  $Q_B$  attend K,V
- 3 candidate C:  $Q_C$  attend K,V

第二，STCA 多层之间共享同一份 K/V：

- 1 Layer 1:  $Q_0$  attend K,V
- 2 Layer 2:  $Q_1$  attend K,V
- 3 Layer 3:  $Q_2$  attend K,V

这样做的原因是：

- 1 K/V 是用户历史事实
- 2 历史事实本身不需要随层数变化
- 3 变化的是我们如何看这些历史，也就是 Q

好处是：

- 1 减少参数量
- 2 减少重复计算
- 3 降低线上 RT
- 4 更适合长序列

## 八、为什么共现序列可能变差

实际业务中确实会构造共现序列，例如：

- 1 和当前 item 经常一起被观看的 item
- 2 同类用户共同消费的 item
- 3 item-item 相似邻居
- 4 图关系邻居
- 5 同主播、同类目、同标签下的高共现 item

这类特征常见于 item-to-item、协同过滤、图推荐、co-attention 等方向。

但共现序列和真实用户行为序列不一样。

真实观播序列具有：

- 1 时间顺序
- 2 用户个人兴趣演化
- 3 行为强度
- 4 上下文连续性

共现序列很多时候更像一个集合或图邻居：

- 1 没有自然时间顺序
- 2 不一定代表当前用户真实兴趣
- 3 可能包含热门 item 或伪相关

STCA 的设计假设是：

- 1 K/V 是用户历史兴趣事实
- 2 Q 是当前 target 对这些历史事实的查询视角

如果序列本身不是用户历史事实，而是共现、相似、图邻居，那么多层 target attention 可能会放大噪声。

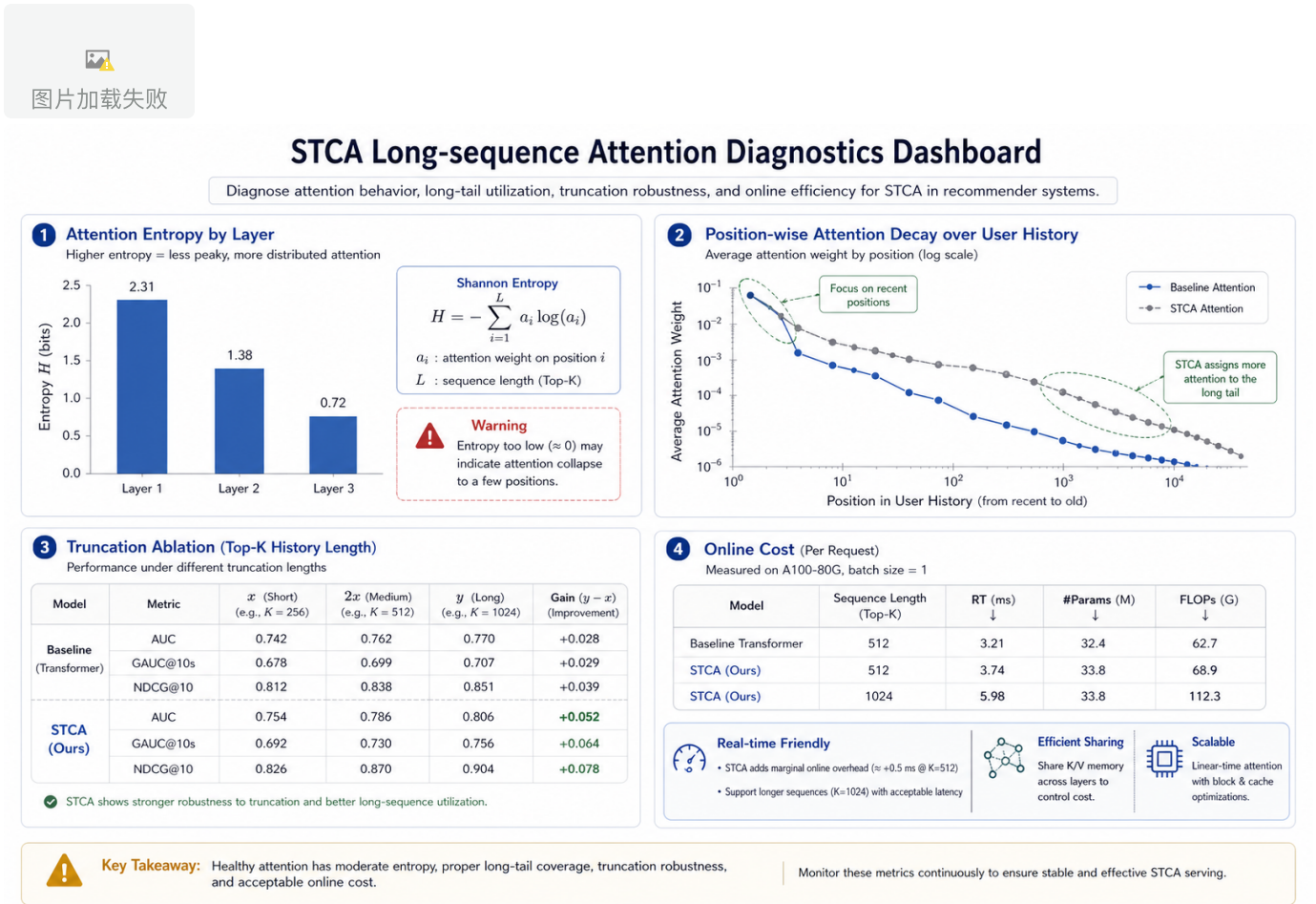
因此建议：

- 1

真实观播序列：适合 STCA
- 2

共现序列：更适合 pooling、graph aggregation、item-to-item attention，不一定适合多层时序 attention

九、注意力分布熵怎么做



$$1 \quad H = -\sum a_i \log(a_i)$$

其中：

1  $a_i$  = 第  $i$  个历史行为的 attention weight

如果 attention 很平均：

- 1 每个历史行为权重差不多
- 2 熵高

如果 attention 很集中：

- 1 少数历史行为权重大
- 2 熵低

PyTorch 伪代码：

```
1 def attention_entropy(attn_weights, mask=None, eps=1e-12):
2     # attn_weights: [batch, heads, seq_len]
3     if mask is not None:
4         attn_weights = attn_weights.masked_fill(~mask[:, None, :], 0.0)
5
6     entropy = -(attn_weights * torch.log(attn_weights + eps)).sum(dim=-1)
7     return entropy.mean(dim=0) # 每个 head 的平均熵
```

建议监控：

- 1 每层平均熵
- 2 每个 head 的熵
- 3 不同序列长度下的熵
- 4 不同用户活跃度下的熵

预期不是“越低越好”，而是：

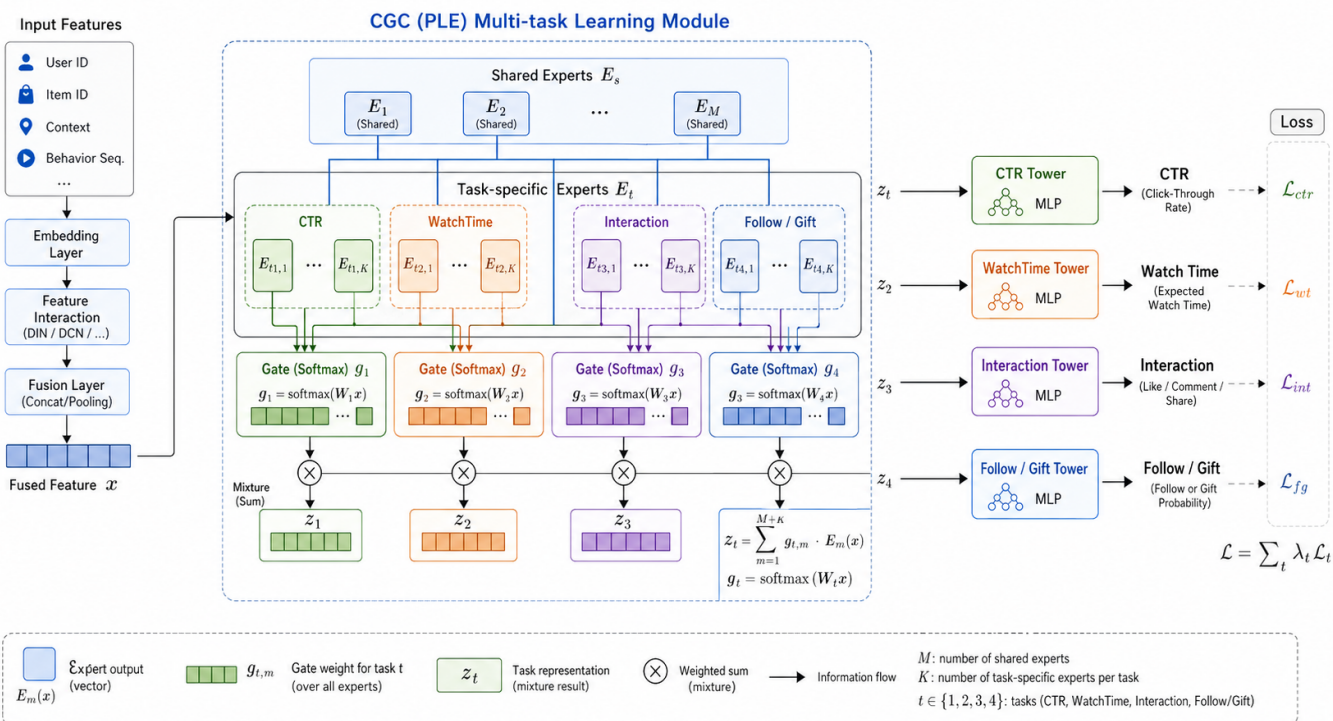
- 1 Layer 1 相对广泛
- 2 Layer 2 开始聚焦
- 3 Layer 3 更聚焦，但不能塌缩到单点

还要结合位置衰减曲线和截断消融判断。

## 十、CGC 和 PLE 是什么关系



图片加载失败



**读图说明：** 图左侧是融合后的特征，包括 STCA 产出的用户兴趣表达。中间是 CGC 模块：有 shared experts，也有 CTR、WatchTime、Interaction、Follow/Gift 等任务专属专家。每个任务通过自己的 softmax gate 选择共享专家和任务专家。公式  $z_t = \sum g_{t,m} E_m(x)$  表示第  $t$  个任务的输入表示是多个专家输出的加权和。

CGC 不是 PLE 的同义词。

关系是：

- 1 CGC 是 PLE 的核心模块
- 2 PLE 是多层堆叠的 CGC

CGC 全称是：

- 1 Customized Gate Control

它来自腾讯 RecSys 2020 的 PLE 论文，用来解决多任务学习中的共享与冲突问题。

假设有多任务：

- 1 点击 CTR
- 2 观看时长 Duration
- 3 互动 Interaction
- 4 关注 Follow
- 5 送礼 Gift
- 6 负反馈 Negative

这些任务之间既有共性，也有冲突。例如：

- 1 点击高不一定时长高
- 2 时长高不一定互动高
- 3 送礼样本极稀疏但业务价值高
- 4 负反馈和正反馈方向相反

CGC 的做法是把专家分成：

- 1 shared experts: 所有任务共享
- 2 task-specific experts: 每个任务专属
- 3 gate: 每个任务自己选择专家组合

CTR 任务的 gate 可以选择：

- 1 CTR 专家 + shared 专家

时长任务的 gate 可以选择：

- 1 时长专家 + shared 专家

互动任务的 gate 可以选择：

- 1 互动专家 + shared 专家

这样既能共享通用信息，又能保留任务差异。

PLE 则是把 CGC 堆多层，让共享信息和任务特有信息逐层分离。

## 十一、作者这个模型可能是什么结构

结合你前面几部分内容，可以推测作者的整体模型大概是：



```

1  输入特征
2    └─ 用户画像
3    └─ 主播/直播间离线特征
4    └─ 实时特征
5    └─ 用户观播长序列
6    └─ candidate 直播间特征
7
8  长序列模块
9    原始长序列 y
10   ↓
11   GSU 根据 candidate 粗筛 Top-K 历史行为
12   ↓
13   ESU / STCA 做多层 target attention
14   ↓
15   target-aware user interest
16
17  特征融合
18   dense/sparse features + STCA interest + realtime tower output
19   ↓
20   shared backbone / token interaction block
21
22  多任务模块
23   CGC / PLE
24   └─ CTR tower
25   └─ WatchTime tower
26   └─ Interaction tower
27   └─ Follow tower
28   └─ Gift tower
29   └─ Negative feedback tower

```

如果和近几年模型趋势对齐：

```

1  STCA: 更偏长用户序列建模
2  RankMixer / TokenMixer: 更偏特征 token mixing 与 ranking backbone 扩容
3  OneTrans: 尝试把非序列特征和序列特征统一到 Transformer
4  CGC / PLE: 解决多任务共享与冲突

```

它们不是同一个模块，但可以组成一个现代工业精排模型。

## 十二、实践建议

### 1. 序列扩长前先证明有价值

先统计：

- 1 用户一周有效观播数分布
- 2 x 条之外历史的类目分布
- 3 近期兴趣和长期兴趣差异
- 4 长序列截断消融

如果 x 条之外的信息和当前任务没有增益，扩充只会增加噪声和 RT。

## 2. STCA 优先用于真实用户行为序列

优先使用：

- 1 真实观播序列
- 2 有效观看序列
- 3 点击序列
- 4 互动序列

谨慎使用：

- 1 共现序列
- 2 相似 item 序列
- 3 图邻居序列

## 3. 层数从 2 层开始

推荐实验顺序：

- 1 1 层 target attention baseline
- 2 2 层 STCA
- 3 3 层 STCA
- 4 4 层 STCA

每次都同时看：

- 1 AUC / GAUC / NLL
- 2 线上 RT
- 3 注意力熵
- 4 位置衰减曲线
- 5 截断消融

## 4. KV 复用要工程化实现

同一个用户请求中，历史 K/V 可以提前算好，多个 candidate 复用，计算完放到内存中或者redis中。

如果线上候选数量多，这个优化会很重要：

- 1 不复用：每个 candidate 重算 K/V
- 2 复用：用户历史 K/V 算一次，多个 Q 共享

## 5. CGC/PLE 要看任务冲突

如果多任务之间冲突明显，比如点击、时长、互动、送礼、负反馈互相拉扯，可以引入 CGC/PLE。

如果任务少、冲突不明显，简单 shared-bottom + task tower 可能就够了。

## 十三、实验与监控清单

建议离线实验：

- 1 1. 序列长度：x / 2x / y
- 2 2. ESU 结构：普通 attention / STCA 2层 / STCA 3层
- 3 3. Q 更新：FFN / SWiGLU
- 4 4. KV：每层独立 / KV 复用
- 5 5. 序列类型：真实观播序列 / 共现序列 / 相似 item 序列
- 6 6. 多任务结构：shared-bottom / CGC / PLE

建议监控指标：

- 1 AUC / GAUC / NLL
- 2 线上 RT / 参数量 / FLOPs
- 3 注意力分布熵
- 4 注意力位置衰减曲线
- 5 远端历史权重占比
- 6 截断消融收益
- 7 多任务 seesaw 指标
- 8 各任务 tower 的 PCOC / 校准

## 十四、总结

这部分工作的核心是：

- 1 用户历史序列从  $x$  扩到  $y$  后, 普通 target attention 不再足够
- 2 因此用 GSU 先粗筛长序列, 再用 STCA 在 ESU 中做多层 target attention
- 3 STCA 用 SWiGLU 增强  $Q$  的更新能力, 用 KV 复用控制参数量和计算量
- 4 最后用 CGC/PLE 把长序列兴趣表达分配给不同任务

一句话概括:

- 1 GSU 解决长序列算不过来, STCA 解决长序列看不透, CGC 解决多任务怎么共享和隔离。

## 参考资料

- [SIM: Search-based Interest Model](#)
- [TWIN: Two-stage Interest Network at Kuaishou](#)
- [TWIN V2: Scaling Ultra-Long User Behavior Sequence Modeling](#)
- [PLE: Progressive Layered Extraction](#)
- [GLU Variants Improve Transformer](#)
- [OneTrans](#)